

Frequently Asked Questions

Revision: 1 **Last modified:** 2026-08-15T14:15:00Z **Audience:** Operators.

Purpose: Practical answers to the questions that come up most often when running boba day-to-day.

Tracker credentials & cookies

Q1. My cookies expired – how do I refresh them?

Every tracker session expires eventually (RuTracker: ~90 days; NNM-Club: weeks; Kinozal: months). Refresh in three steps:

1. **Re-login in your browser** to the tracker (solve any CAPTCHA, tick *Remember me*, land on your profile page).
2. **Re-export the cookies** to `~/Downloads/cookies_<tracker>.txt` – see the step-by-step in [docs/guides/tracker-credentials.md](https://docs.bobaproxy.com/guides/tracker-credentials.md).
3. **Trigger the reload:**

```
bash scripts/boba-svc.sh restart
```

The `boba-svc restart` hook auto-invokes the cookie loader; the containers pick up the fresh cookies on the next up. You can also invoke the loader directly (`bash scripts/load-tracker-cookies.sh`) without restarting, then `docker exec / podman restart qbittorrent-proxy` when convenient.

Q2. Why does RuTracker keep failing auth?

Almost always one of these three (in order of frequency):

1. **The session cookie expired.** Re-export per Q1. `cookies_rutracker.txt` MUST contain a `bb_session=` line for `.rutracker.org` (case-insensitive). The challenge script's Check 4a spells this out:

```
RUTRACKER_COOKIES contains bb_session= (RuTracker session cookie present)
```

No `bb_session` – not a logged-in export – RuTracker WILL refuse auth.

2. **Cloudflare challenged and got no `cf_clearance`.** RuTracker fronts with Cloudflare periodically; the loader captures the full jar including `cf_clearance`. If Cloudflare re-challenges you in the browser, re-export AFTER solving it.

3. **RuTracker credentials env var is stale.** RUTRACKER_USERNAME / RUTRACKER_PASSWORD in .env are the FALLBACK auth path when the cookie is absent. If your password changed on the site, update it in .env too.

Q3. How does the system find my cookies file?

The convention is fixed:

```
${TRACKER_COOKIE_DIR:-$HOME/Downloads}/cookies_<tracker>.txt
```

- <tracker> is **lowercase**: rutracker, nmclub, rutor, kinozal.
- Default source dir is \$HOME/Downloads on the host running the containers.
- The loader (scripts/load-tracker-cookies.sh) is auto-invoked before every boba-svc up, boba-svc restart, install.sh Stage 6, and start.sh boot â€” you never need to remember to run it manually.
- Nothing scans the file otherwise. If you name it cookies_ru_tracker.txt or rutracker_cookies.txt, the loader ignores it â€” the naming is exact.

Q4. Can I put cookies files somewhere other than ~/Downloads?

Yes, two equivalent ways:

- **Env var** (persistent â€” export in your ~/.bashrc / ~/.zshrc):

```
export TRACKER_COOKIE_DIR="/mnt/usb/browser-export"
```

- **Flag** (one-shot):

```
bash scripts/load-tracker-cookies.sh --dir /mnt/usb/browser-export
```

--dir wins if both are set. The wiring hooks pass no --dir flag, so they honour TRACKER_COOKIE_DIR â€” meaning an exported env var affects auto-load too.

Q5. What if two files exist for the same tracker (e.g. cookies_rutor.txt and cookies_rutor.txt.bak)?

The loader only reads the canonical name cookies_<tracker>.txt. Any suffix (.bak, .old, .2) is IGNORED â€” no ambiguity, no picking-the-wrong-file class of bug. Deleting or renaming .bak copies is optional; theyâ€™re just harmlessly ignored.

Q6. Are cookies files ever committed to git?

No, three defences:

1. ~/Downloads lives on the operatorâ€™s host filesystem, outside the repo entirely â€” git status in the boba repo never sees them.

2. Even inside the repo, `.env` (where the parsed cookie header ends up) is `.gitignore-matched` (checked by `challenges/scripts/credentials_wired_challenge.sh` Check 2).
3. The loader runs a **§11.4.10.A pre-store leak audit** on every non-trivial cookie value `"git grep -F on tracked files + git log -S on recent history`. If a value ever appears in the repo, the store is **BLOCKED** and the operator is instructed to rotate the cookie in-browser (invalidate the session server-side), re-export, re-run.

Cookie **values** additionally never enter any log line `"` the loader prints only tracker name, cookie count, cookie NAMES, and file mtime.

Q7. Do I need cookies for RuTor? It's a public tracker.

Not for the auth path (RuTor has no login endpoint). But an exported `cookies_rutor.txt` is still useful because the jar carries `cf_clearance` `"` Cloudflare's proof-of-work challenge token `"` which lets scraped queries bypass rate limits and challenges. If RuTor searches start returning HTML challenge pages instead of results, export a fresh cookies file after solving the Cloudflare challenge in your browser.

Q8. Can I mix cookies-file + username/password for the same tracker?

Yes. `.env` can hold both `<TRACKER>_USERNAME / <TRACKER>_PASSWORD` and `<TRACKER>_COOKIES` at the same time. The plugins prefer the cookie when it's present + valid; the credentials are the fallback if the cookie fails mid-session. This is the recommended posture for RuTracker and Kinozal.

Q9. The loader shows `parse_err=1` `"` what does that mean?

The cookie file exists but the delegated `scripts/extract-tracker-cookies.sh` couldn't find the required session cookie for that tracker. Almost always the export was done while un-logged-in, or the operator exported the wrong site's cookies (e.g. saved `cookies_rutracker.txt` from a session where they were only on `rutracker.net` mirror without logging in). Re-login `"` re-export.

Q10. The loader shows `blocked=1` `"` what does that mean?

The §11.4.10.A leak audit found the cookie value in tracked repo files or recent git history. The store is refused because writing it would silently re-leak an already-compromised secret. **Rotate the cookie in your browser** (log out + log back in, or hit any per-session `"` invalidate all sessions `"` control the tracker exposes), re-export `cookies_<tracker>.txt`, and re-run the loader. The output names the file(s) / commit(s) that hit `"` investigate those separately to figure out how the leak happened (usually an accidental commit long ago; the §11.4.10.A audit is the safety net).

Startup & runtime

Q11. Does boba-svc restart really refresh cookies?

Yes. Since 2026-08-15 both `_cmd_up` and `_cmd_restart` in `scripts/boba-svc.sh` invoke the cookie loader BEFORE `systemctl --user start boba.target`. If the loader fails, a WARNING surfaces (per [Â§11.4.234](#) always-unblocked) and the target still starts “ with the previous `.env` values intact.

Q12. start.sh also refreshes cookies?

Yes, at the top of `main()`, right after arg parsing and before `load_environment`. Same failure semantics: warning-not-blocking.

Q13. What are the exit codes of the loader?

Code	Meaning
0	Success “ every file loaded, unchanged, or absent.
1	Â§11.4.10 .A leak audit failure “ store blocked.
2	Parse error “ a file failed the required-session-cookie check.
3	Invocation error “ bad flag, missing arg, extractor absent.

Wiring hooks tolerate ANY non-zero exit “ the loader is advisory, never a boot gate.

Related documentation

- [docs/guides/tracker-credentials.md](#) “ the operator manual covering both env-var and cookies-file mechanisms.
- [docs/scripts/load-tracker-cookies.md](#) “ the full script-level reference for the loader.
- [docs/scripts/extract-tracker-cookies.md](#) “ the single-tracker extraction primitive.
- [docs/TOKENS_AND_KEYS.md](#) “ the full env-var reference.
- [CLAUDE.md](#) “ project-wide policy anchors.